1. INTRODUCTION

• Project Title: Product Inventory Management System(Backend)

• Team ID:SWTID-2026-4269

• Team Leader: P.Suchitra& suchitrasuji@gmail.com

• Team Members:

S.Sridharshini& sridharshinisaravanan2007@gmail.

T.Swetha&swethajeeva2006@gmail.com @gmail.com

S.Sumaya& sumiyashakul@gmail.com

PROJECT OVERVIEW:

The Product Inventory Management System is a backend-focused web application developed using Node.js, Express.js, and MongoDB, designed to provide a secure and scalable solution for managing product data through RESTful APIs. The system enables users to create accounts, log in securely, and access protected routes using JWT-based authentication, ensuring that only authorized users can perform operations. It supports complete CRUD functionality, allowing users to add new products, view their inventory, update product details such as price or quantity, and delete items when needed.

Key Objectives:

* Provide secure JWT-based user authentication and authorization for protected API access.

* Enable efficient product inventory management through RESTful CRUD operations.

* Ensure user-specific data isolation so each user can manage only their own products.

* Implement scalable backend architecture using Node.js, Express.js, and MongoDB.

* Deliver fast and reliable API responses with proper validation and error handling.

Target Users:

* Small business owners managing product stock and inventory records.

* Freelancers or sellers tracking items or services in a structured way.

* Developers learning backend API development and authentication workflows.

* Students working on backend or full-stack development projects.

* Anyone needing a secure and simple backend system for managing product data.

PROJECT INSTRUCTION

• Create backend project folder • Configure server using Express.js • Setup MongoDB database connection • Implement authentication APIs • Implement product CRUD APIs • Test APIs using Thunder Client in VS Code

PRE-REQUISITES

Software

Purpose

Node.js

JavaScript runtime for backend

MongoDB

Database storage

npm

Package manager

VS Code

Development environment

Knowledge Requirements

• JavaScript ES6 concepts

• Node.js and Express.js REST API development

• MongoDB and Mongoose ODM

• JWT authentication

• Middleware architecture

Project Thumbnail:

PROJECT ARCHITECTURE

High-level architecture diagram illustrating the three-tier backend structure.

Node.js + Express Backend

* RESTful API architecture

* JWT authentication middleware

* MVC pattern (Models, Controllers, Routes)

* Input validation and error handling

MongoDB Database

*Document-based NoSQL storage

*Mongoose ODM for schema modeling

*Two collections: Users and Contacts

*Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

Backend Architecture (Node.js + Express)

- Controllers: Handle authentication and product logic
- Models: Define database schema
- Routes: API endpoints for authentication and products
- Middleware: JWT verification and error handling
- Configuration: Environment variables and database connection

Database Architecture (MongoDB)

Users Collection:

- _id • name • email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId (reference to user)
- name
- price
- quantity
- description
- timestamps

Relationship:

One User → Many Products

Indexes:

- email (unique)
- userId
- product name

ER diagram showing User → Product relationship:

Features:

Description

User Authentication

JWT-based register and login system

Product CRUD

Add, view, update and delete products

Private Data Access

Users access only their own products

Product Search

Search products by name

Secure API

Middleware protected routes

Roles and Responsibility

User:

Registration: Register using name, email and password

Login: Login using registered credentials

Product Add: Create product with name, price, quantity and description

Edit Product: Update existing product

Delete Product: Remove product from inventory

Open terminal inside the server folder and run:

Package Purpose:

express → Backend framework

mongoose → MongoDB connection

cors → Cross-origin requests

dotenv → Environment variables

jsonwebtoken → Authentication

bcryptjs → Password hashing

Create Environment File:

Create a .env file in the root of the server folder.

Example:

```
PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key
```

Backend Development

Setup Express Server:

Inside src/server.js, create the basic Express server.

Create Controllers

Inside: src/controllers/

Auth Controllers

The Auth Controller manages all user authentication-related operations in the system. It handles user registration and login functionality while ensuring secure access using JWT tokens.

Responsibilities:

Register new users

Validate user input data

Hash passwords using bcrypt before storing

Authenticate users during login

Generate JWT tokens for authorized access

Send appropriate success and error responses

Main Functionalities:

Register User: Creates a new user account after validating email uniqueness and encrypting the password.
Login User: Verifies credentials and returns a JWT token for protected API access.

Product Controller

The Product Controller handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

Add new products to inventory

Fetch products belonging to logged-in users

Update product details like price, quantity, or description

Delete products securely

Perform product search functionality

Main Functionalities:

Create Product: Adds a new product with a reference to the authenticated user.

Get Products: Retrieves only the products created by the logged-in user.

Update Product: Modifies product details while verifying ownership.

Delete Product: Removes a product safely from the database.

Search Product: Allows filtering products based on name.

Create Middleware

Inside: src/middlewares/

Create:auth.middleware.js

Auth Middleware (protect)

The Auth Middleware secures protected routes by verifying the JSON Web Token (JWT) sent in the request header. It checks for a valid Authorization header starting with "Bearer " and validates the token using the secret key. If the token is valid, the decoded user information is attached to req.user and the request proceeds to the next controller. If the token is missing or invalid, the middleware returns a 401 Unauthorized response, preventing access to private APIs.

Create Routes

Inside: src/routes/

auth.routes.js

Auth Routes:Auth routes handle user authentication by providing endpoints for registration and login. These routes connect incoming requests to the Auth Controller to create new users and generate JWT tokens for secure access.

product.routes.js

Product Routes:

Product routes manage all inventory-related operations such as creating, retrieving, updating, and deleting products. These routes are protected using authentication middleware to ensure only authorized users can access their own product data.

Connect Routes to Server

Import all routes into server.js

These route configurations connect the authentication and product APIs to the main Express server using versioned endpoints. The /api/v1/auth path handles all user authentication requests, while /api/v1/products manages product-related operations, helping maintain a structured and scalable API architecture.

Database Configuration and Setup

Create MongoDB Atlas cluster

Add connection string to .env

Connect DB before API testing.

Database Connection:

Create a file inside: src/db/db.js

Purpose: Connect backend to MongoDB using mongoose.

The connectDB function establishes a connection between the application and MongoDB using Mongoose and environment-based configuration. It ensures the server starts only after a successful database connection and logs an error message if the connection fails.

Create Models:

Inside src/models/ create:

User.js

Product.js

User Model:

The User schema defines the structure for storing user details such as name, email, and hashed password in the database. It supports secure authentication and maintains timestamps for record tracking.product Model:

Product Schema:

The Product schema represents inventory items and includes fields like name, price, quantity, description, and a userId reference. It ensures each product is linked to its owner, enabling private data access and management.

Project Execution

Execution Steps

Open terminal in server folder.

Make sure you have a start script in the package.js file.

Run:

npm start

Console should display:

Server running on PORT

Database Connected

API Testing :

Use Thunder Client to test APIs:

Test APIs:

Register User:The register functionality allows new users to create an account by submitting their details, which are validated and securely stored with a hashed password. It ensures unique user creation before granting access to the system.

Login User:The login functionality verifies user credentials and generates a JWT token upon successful authentication. This token is used to access protected routes securely.

Create Product:

Allows an authenticated user to add a new product to their inventory by submitting product details. The product is saved with the user's ID to maintain ownership.

Get all Products:

Fetches all products that belong to the logged-in user. It ensures users can view only their own inventory data.

Update Product:

Enables users to modify existing product details such as name, price, or quantity. The system verifies ownership before allowing any updates.

Delete Product:

Allows users to permanently remove a product from their inventory. The operation is protected to ensure only the product owner can delete it.